

Problem

The Philippines is highly exposed to frequent natural hazards, experiencing an average of twenty tropical cyclones annually. When severe typhoons strike, infrastructure collapse often severs internet and cellular connections, rendering centralized, cloud-based early warning systems inaccessible to vulnerable communities precisely when they are most needed. Additionally, local communities struggle to translate broad national weather data into actionable, street-level evacuation protocols, and there is a lack of affordable, localized water-level sensors to provide real-time alerts.

How the System Resolves the Problem

1. Connectivity failure → the system does not depend on connectivity

A warning system that requires the internet is a warning system that switches itself off during the emergency. The system is therefore built as a **local-first application** rather than a client to a cloud service: the phone holds its own copy of the data and computes its own answers, and the server is an optional enhancement rather than a prerequisite.

- **The map opens and works with no connection**, from downloaded offline tile packs. GPS positioning needs no network, so the user's location is available even in a total blackout.
- **Every report and SOS is stored on the device the moment it is created** — it is immediately useful to the person who wrote it, and is transmitted later, automatically, whenever a path becomes available.
- **When the network is gone entirely, reports and rescue requests travel phone-to-phone** over Bluetooth and Wi-Fi Direct, hopping between nearby devices until one of them reaches connectivity and uploads on everyone's behalf. The mesh only has to reach *one* connected device, not the server — one person driving to higher ground can carry an entire neighborhood's data out with them.
- **Where the data network is down but the cellular network survives**, compact SMS carries reports and rescue requests, which also reaches feature phones that cannot run the app at all.
- **Notifications are triggered locally**, by the device evaluating its own data against the user's watched locations, so alerts continue to fire with no push server and no signal.

The result: the system's capability degrades in fidelity rather than switching off. Full data connection, then SMS, then phone-to-phone relay, then a device that still holds a usable map and can still call for help.

2. Resolution failure → the community produces the street-level layer

National forecasts are accurate but coarse. They can tell a resident that a storm signal has been raised over their province; they cannot tell them whether the road to their child's school is passable. That last mile of information exists only in the heads of the people standing in the water — so the app's job is to collect it, structure it, and give it back.

- **Reports are attached to specific roads and areas**, turning a provincial advisory into a street-level picture.
- **Severity is expressed as a decision, not a measurement.** “Impassable for cars” and “waist-deep” are things an ordinary person can both report accurately and act on immediately, without interpreting rainfall figures.
- **Confirm and dispute, plus a visible confidence indicator and automatic expiry**, mean the map communicates not just what is being reported but *how much to trust it and how fresh it is* — the difference between usable local knowledge and rumor.
- **Radius and saved-route notifications** convert the shared map into a personal, actionable warning: not “a storm is coming” but “the road you take home is now impassable.”
- **Official PAGASA and NDRRMC updates sit alongside the community layer**, so authoritative national guidance and local ground truth are read together rather than in separate places.

3. Sensor scarcity → the community is the sensor network

Dedicated water-level sensors are expensive, and a budget large enough to instrument every chokepoint in a municipality does not exist at barangay level. The app sidesteps the constraint rather than trying to fund its way around it: **every resident with a phone becomes a distributed water-level sensor, at no hardware cost.**

- **The body and vehicle water-level scales are the measurement instrument.** They are deliberately coarse because coarse readings are what untrained people can produce reliably and consistently — and “a car cannot pass” is more directly actionable than a depth in centimeters anyway.
- **Coverage beats precision here.** A funded sensor program might instrument a handful of fixed points; residents are already everywhere, including the streets no budget would ever cover.
- **Confirm/dispute and staleness expiry act as the calibration layer**, filtering error and out-of-date readings the way a sensor network relies on maintenance and validation.

The honest trade-off: human reports are *episodic* rather than continuous, and depend on someone being present and willing to report. Physical sensors would complement this well in future. But for the coverage a community can achieve today, at a cost a barangay can actually afford, people are the only sensor network available — and this app is what turns them into one.

Features

1. Interactive Flood Map

The primary surface of the app, and the first thing a user sees. It answers the question a resident actually faces during a storm: *is this specific street passable right now?*

- **Base map** (OpenStreetMap or Google Maps SDK) showing the user's live location.
- **Crowdsourced flood markers and overlays** applied to roads and areas, so a flooded road reads as a flooded road rather than a pin floating over one.
- **Severity levels**, three tiers:
 - Passable with caution
 - Impassable for cars
 - Impassable for all
- **Timestamp of the last report** on every marker, so users can judge how current the information is.
- **Confidence indicator** derived from the number of confirming reports — distinguishing a single unverified sighting from a road four neighbors have independently confirmed.
- **Filtering** by severity, by recency, or by barangay/district.

2. Crowdsourced Flood Reporting

The map is only as good as the reports feeding it, so reporting has to be fast enough to do in the rain, one-handed, in the dark.

- **Report a flooded road or area** by tapping a location on the map or using the current GPS position.
- **Water-level estimate**, in whichever frame of reference the reporter finds natural:
 - Body-referenced — ankle / knee / waist / chest
 - Vehicle-referenced — truck / car / motorcycle / not passable
- **Optional photo** attached to the report.
- **Automatic timestamp** on submission.
- **Confirm or dispute existing reports** — Waze-style trust verification, where the community itself validates what it sees. Reports gain credibility through corroboration rather than asserting it.
- **Automatic expiry**, with reports flagged as *stale* after a set time unless someone re-confirms them. Floodwater changes fast, and information that was true an hour ago can be dangerous now.

3. Real-Time Notifications

Users should not have to open the app to learn that something changed near them.

- **New flood report within a user-defined radius** of their home or current location, or along a saved route.

- **A previously flooded area near them marked as cleared**, so people are not stranded by information that is out of date in the other direction.
- **Official PAGASA/NDRRMC updates** delivered when connectivity allows.
- **Graceful offline degradation** — notifications must continue to function when the network does not, which is handled by the offline-first design below rather than treated as an exception.

4. SOS / Rescue Request

The feature that matters most on the worst night, and the one that most needs to work without infrastructure.

- **One-tap “I need rescue”** carrying live GPS position, with optional context: number of people, medical needs, current water level.
- **Routing to help** — to nearby rescuers, to barangay responders, and to a shared rescue-request board visible to volunteers and the LGU.
- **Status updates** visible to the person who sent it: *received* → *en route* → *rescued*. Someone waiting on a roof deserves to know whether anyone has seen their request.
- **Peer-to-peer broadcast to nearby app users** when there is no server connectivity at all, so a rescue request can still travel even with every tower down.

5. Offline-First Design

This is the core design constraint of the project, not an add-on. Every feature above must remain useful when the network fails, because that is precisely when they are needed.

- **Downloadable offline map tile packs** per region, so the map opens and works with no connection.
- **Local storage with queue-and-sync** — flood reports and SOS requests are stored on the device immediately and transmitted automatically once connectivity returns.
- **Graceful low-bandwidth and intermittent handling** — compression and retry with backoff, so a weak or flapping connection is used efficiently rather than wasted.
- **Alternative transport when cellular and internet are fully down:**
 - **Bluetooth / Wi-Fi Direct mesh relay** — reports and SOS requests hop device-to-device between nearby phones until one of them reaches connectivity and uploads on everyone’s behalf.
 - **SMS-based fallback** for reporting and alerting, covering feature phones and data-down scenarios.
- **Minimized battery and data usage**, since power outages routinely outlast the storm itself and a phone that dies is a user the system can no longer reach or help.